Design and Analysis of Algorithm

Lecture 18: Doubly Linked List

Dr. Tauheed Ahmed

Content

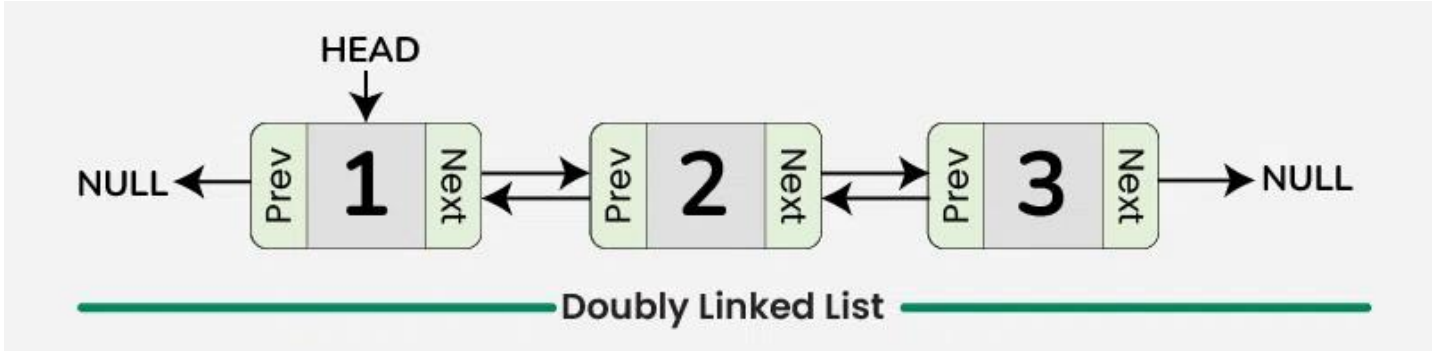
- Introduction
- Operations

Doubly Linked List

A doubly linked list comprises a set of nodes, each node having a pointer to the next node in the list AND a pointer to the previous node. We keep a pointer to the first node in a list head pointer.

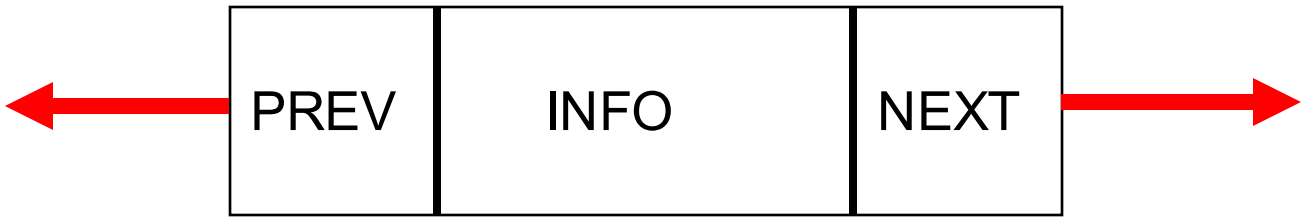
In a data structure, a doubly linked list is represented using nodes that have three fields:

1.Data
2.A pointer to the next node (**next**)
3.A pointer to the previous node (**prev**)



Node

```
typedef struct _node {
    int contents ;
    struct _node *next ;
    struct _node *prev;
} node ;
```




```
//Create the node
    struct node* n = malloc(sizeof(struct node));
    n->contents = 800;
    n->next = NULL; n->prev = NULL;
```

Doubly Linked List: Traversal

Traversal in a Doubly Linked List involves visiting each node, processing its data, and moving to the next or previous node using Traversal.

Forward Traversal

```
void forwardTraversal(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
       temp = temp->next;
    }
}
```

Backword Traversal

```
void backwordTraversal(struct Node* head) {
    struct Node* temp = tail;
    while (temp != NULL) {
       temp = temp->prev;
    }
}
```

Insertion In Doubly Linked List

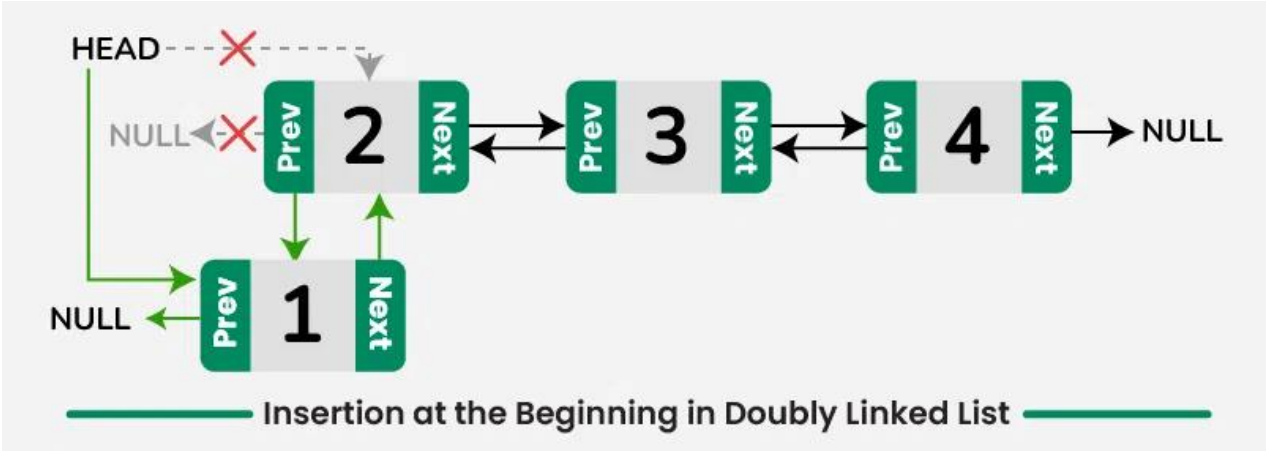
There are different situation for inserting element in Doubly linked list.

- Insertion at the Beginning in Doubly Linked List
- Insertion after a given node in Doubly Linked List
- Insertion before a given node in Doubly Linked List
- Insertion at the End in Doubly Linked List

Insertion In Doubly Linked List

Insertion at the Beginning in Doubly Linked List

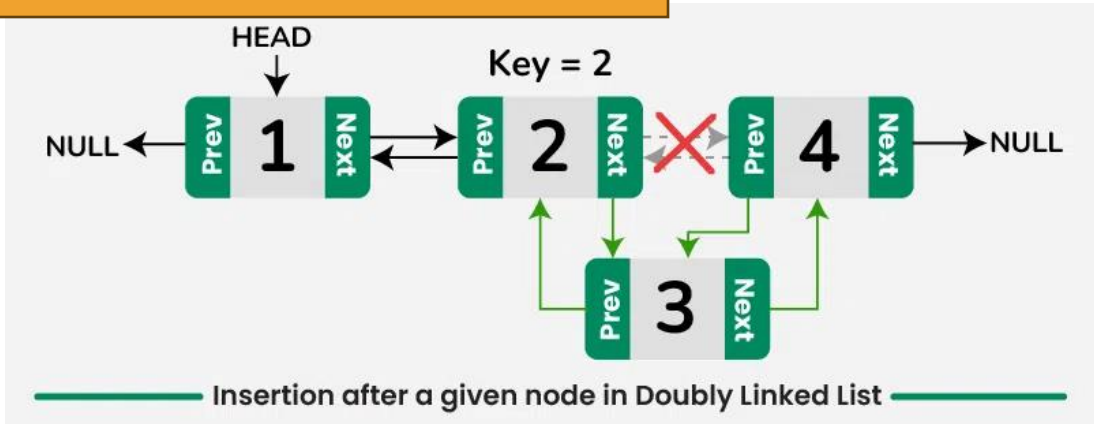
- Create a new node, ***new_node*** with its previous pointer as NULL.
- Set the next pointer to the current head,
- Check if the linked list is not empty then we update the previous pointer of the current head to the new node,
- Finally, we return the new node as the head of the linked list.



Insertion In Doubly Linked List

Insertion after a given node in Doubly Linked List

- Find the given node in the linked list, ***curr***.
- Once we find it, create a new node ***new_node*** with the given input data.
- Set the new node's previous pointer to given node,
- Set the new node's next pointer to the given node's next,
- Then, we update the next pointer of given node with new node,
- Also, if the new node is not the last node of the linked list, then update previous pointer of new node's next node to new node,

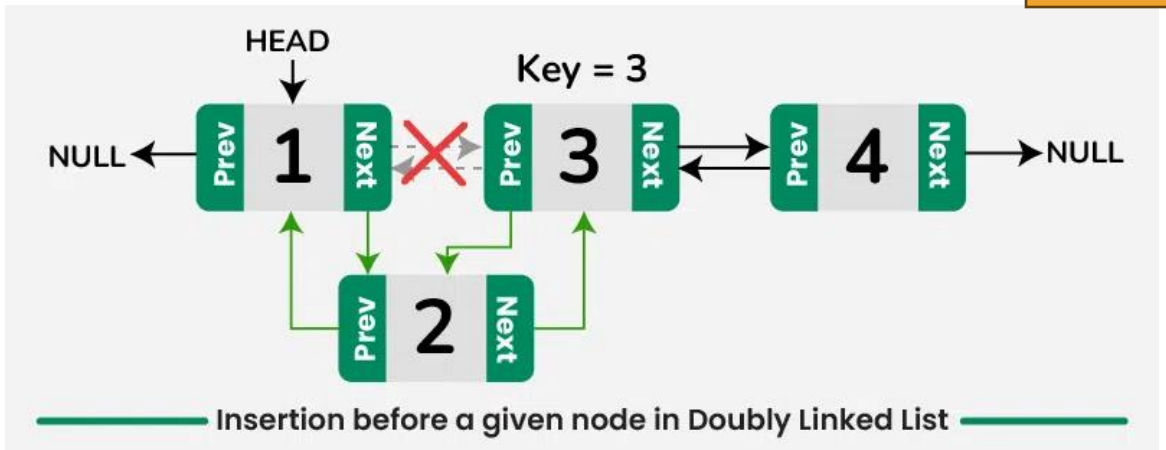


Insertion after a given node in Doubly Linked List

Insertion In Doubly Linked List

Insertion before a given node in Doubly Linked List

- Find the given node in the linked list, ***curr***.
- Once we find it, create a new node say ***new_node*** with the given input data.
- Set the new node's previous pointer to the node before the given node,
- Set the new node's next pointer to the given node,
- Check if the given node is not the head of the linked list, then update next pointer of new node's prev node to new node,
- Finally, we update the previous pointer of given node with new node,

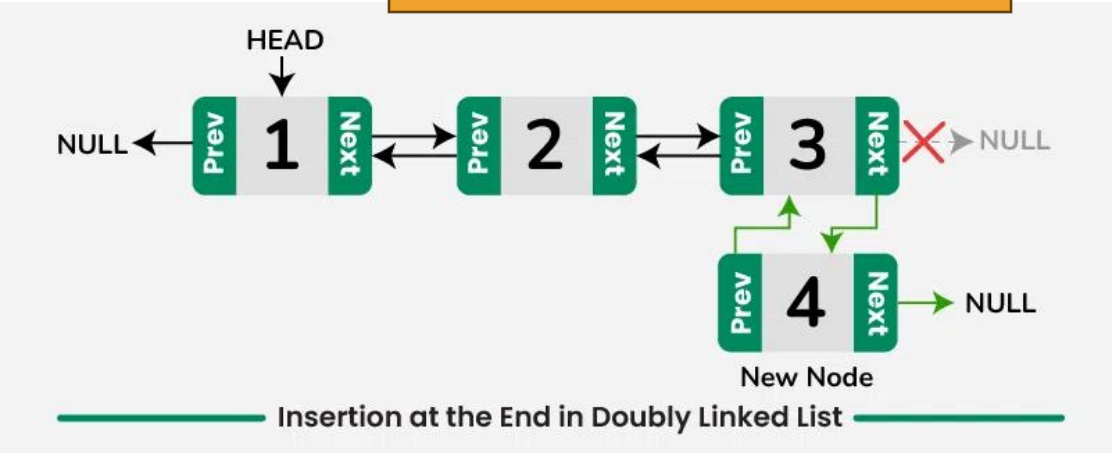


Insertion before a given node in Doubly Linked List

Insertion In Doubly Linked List

Insertion at the End in Doubly Linked List

- Allocate memory for a new node, say ***new_node*** and assign the provided value to its data field.
- Initialize the next pointer of the new node to NULL, ***new_node->next = NULL***.
- If the linked list is empty, we set the new node as the head of linked list and return it as the new head of the linked list.
- Traverse the entire list until we reach the last node, say ***curr***.
- Set the next pointer of last node to new node,
- Set the prev pointer of new node to last node,



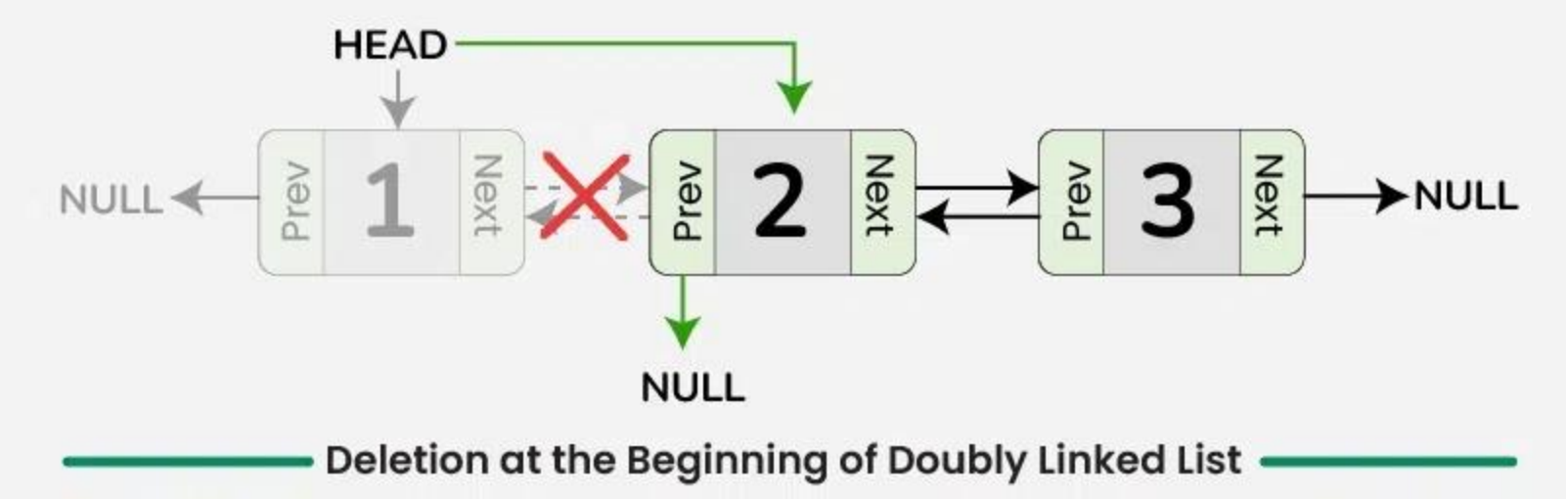
Deletion In Doubly Linked List

- Deletion at the Beginning in Doubly Linked List
- Deletion after a given node in Doubly Linked List
- Deletion before a given node in Doubly Linked List
- Deletion at the End in Doubly Linked List

Deletion In Doubly Linked List

Deletion at the Beginning in Doubly Linked List

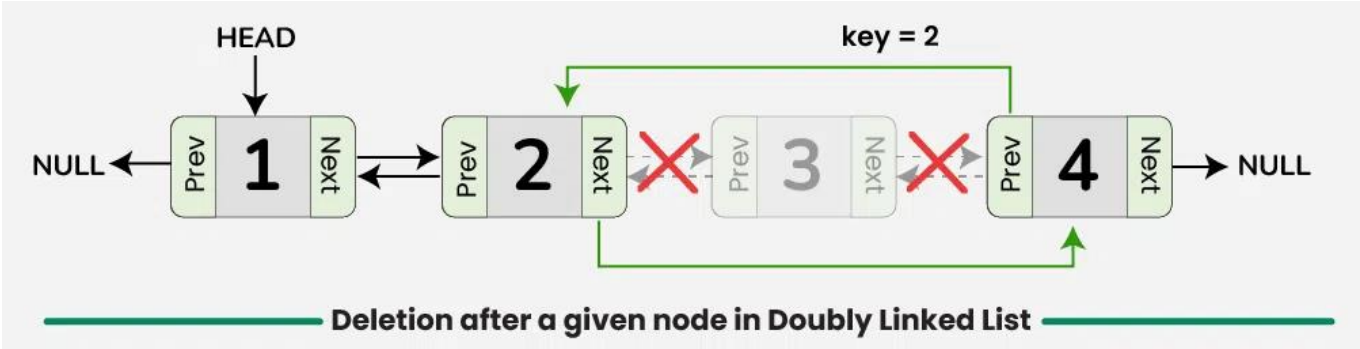
- Check if the list is empty, there is nothing to delete, return.
- Store the head pointer in a variable, say temp.
- Update the head of linked list to the node next to the current head,
- If the new head is not NULL, update the previous pointer of new head,



Deletion In Doubly Linked List

Deletion after a given node in Doubly Linked List

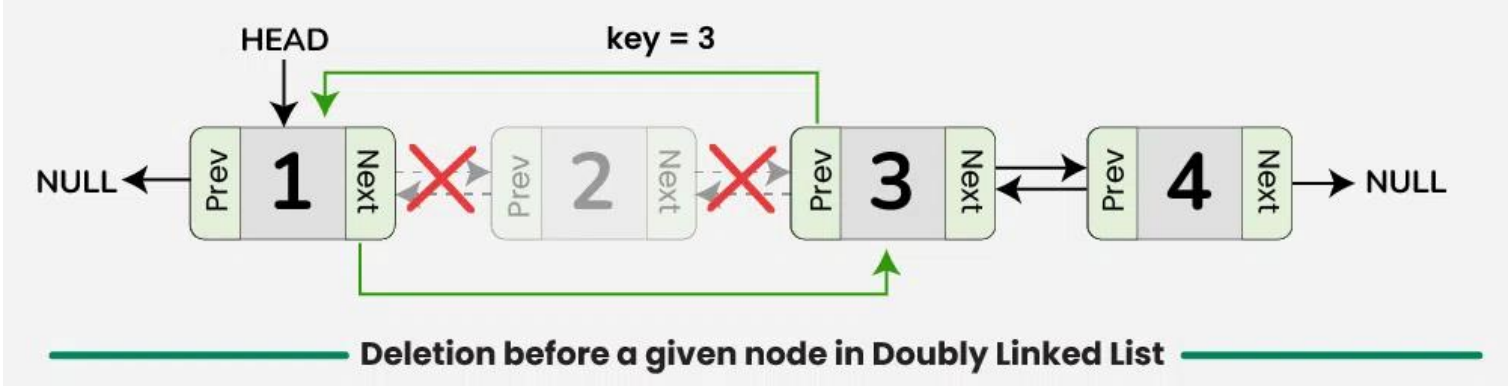
- Initialize a variable **curr** which points to the node with the key value in the linked list.
- if found , check if **curr->next** is not **NULL**.
  - If it's **NULL** then there is no node to delete , return.
  - else , set a pointer **ptr** to **curr->next**, which is the node to be deleted.
  - Update **curr->next** to point to **ptr>next**.
  - If **ptr->next** is not **NULL**, update the previous pointer of **ptr->next** to **curr**.
- Delete **ptr node** to free memory.



Deletion In Doubly Linked List

Deletion before a given node in Doubly Linked List

- Initialize a variable , say **curr** points to the node with the **key** value in the linked list.
- if found , check if **curr->prev** is not **NULL**.
  - If it's **NULL**, the node to be deleted is the **head node**, so there is no node to delete before it.
  - else , set a pointer **ptr** to **curr->prev**, which is the node to be **deleted**.
  - Update **curr->prev** to point to **ptr ->prev.**
  - If **ptr ->prev** is not **NULL**, update **ptr->prev->next** point to **curr**.
- Delete **ptr node** to free memory.

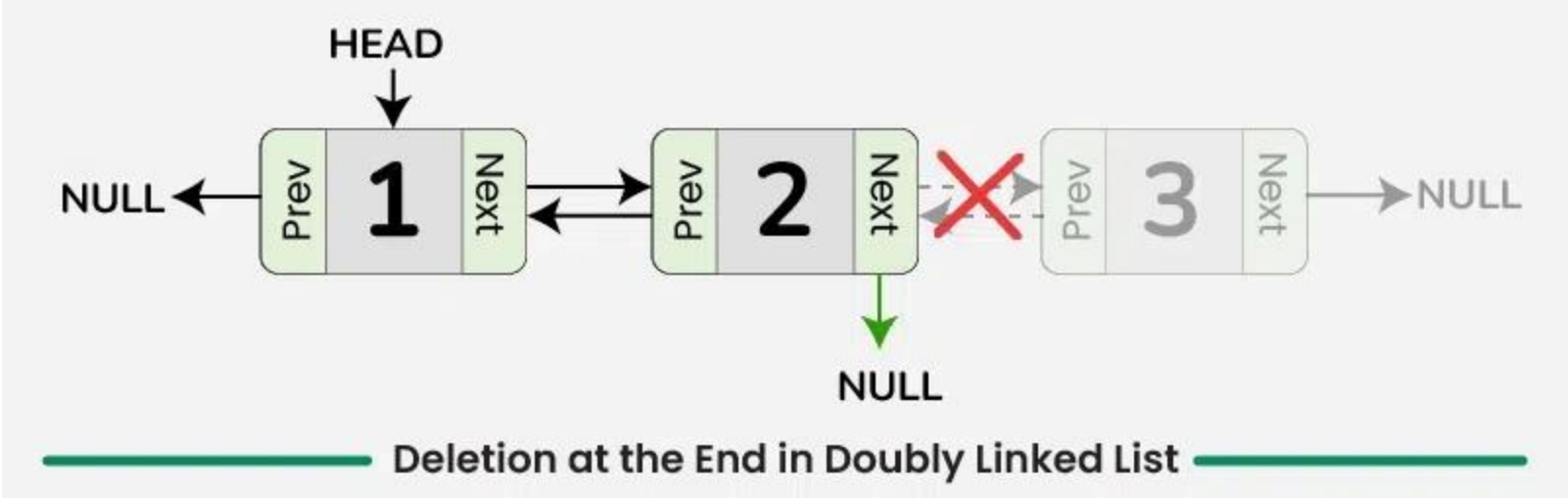


Deletion before a given node in Doubly Linked List

Deletion In Doubly Linked List

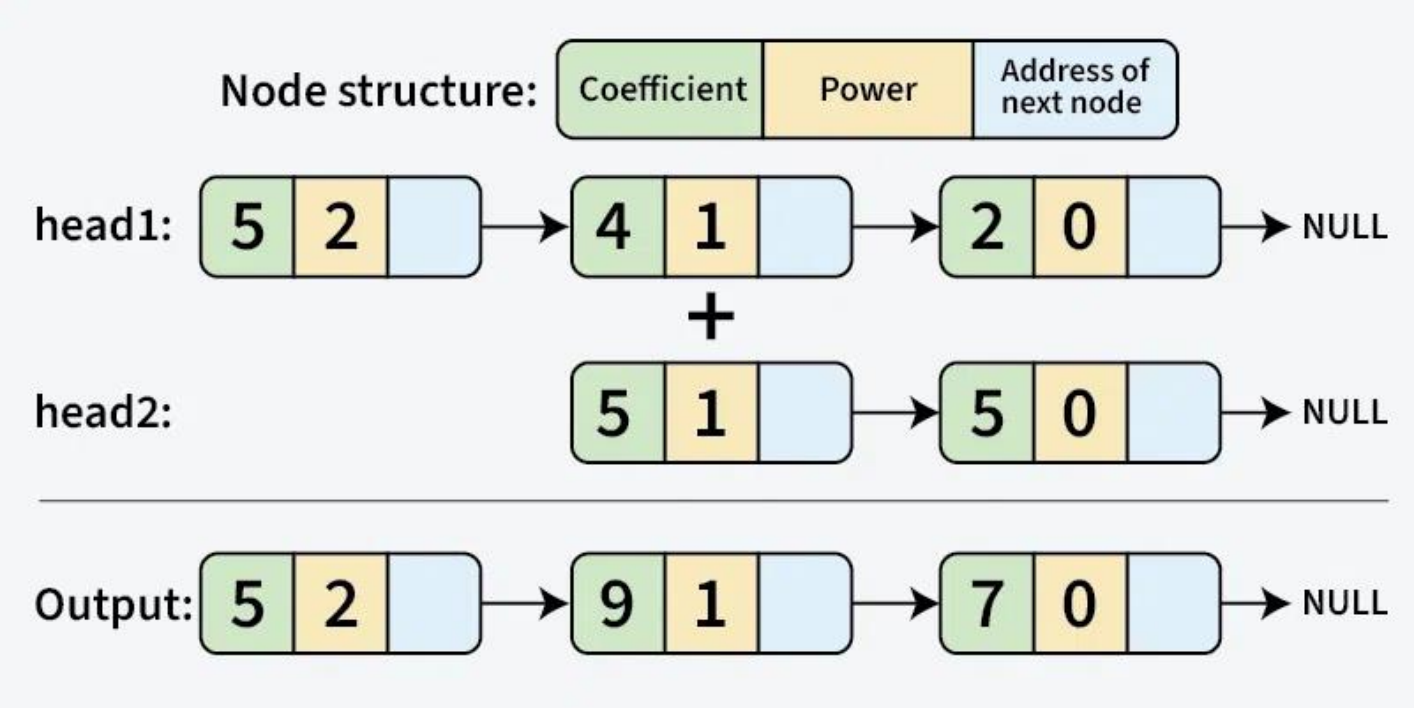
Deletion at the End in Doubly Linked List

- Check if the doubly linked list is empty. If it is empty, then there is nothing to delete.
- If the list is not empty, then move to the **last node** of the doubly linked list, say **curr**.
- Update the **second-to-last** node's next pointer to **NULL**, **curr->prev->next = NULL**.
- Free the memory allocated for the node that was deleted.



Polynomial Addition

Given two polynomial numbers represented by a linked list. The task is to add these lists meaning the coefficients with the same variable powers will be added.



Polynomial Addition

Steps

Step 1: Define Node Structure

- Each node contains three fields:
  - **coefficient** (numerical value of the term)
  - **exponent** (power of the variable)
  - **next** (pointer to the next node)

Step 2: Create a Function to Insert a Node

- If the linked list is empty, insert the new node as the head.
- If the exponent already exists, add the coefficients.
- Otherwise, insert the new node in decreasing order of exponent values.

Polynomial Addition

Steps

Step 3: Traverse Two Polynomial Linked Lists and Add Terms

- **While both polynomials are not empty**
  - If the exponent of **poly1** is greater than **poly2**, insert **poly1's term** into the result list.
  - If the exponent of **poly2** is greater than **poly1**, insert **poly2's term** into the result list.
  - If both have the same exponent, add their coefficients and insert the sum into the result list (only if the sum is nonzero).
  - Move to the next term in both lists.

Step 4: Append Remaining Terms

- If there are remaining terms in **poly1**, insert them into the result list.
- If there are remaining terms in **poly2**, insert them into the result list.

Step 5: Display the Resultant Polynomial

- Traverse the result linked list and print it in a readable format.